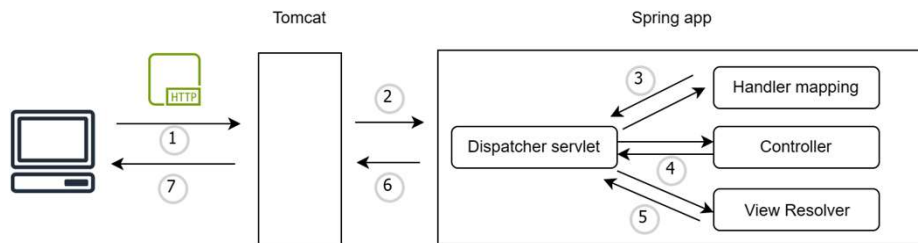


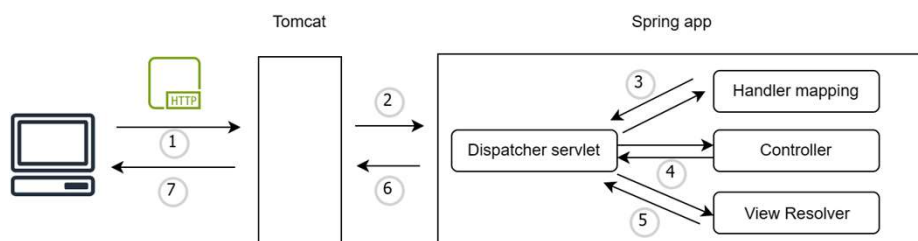
The Spring MVC architecture



1. The client makes an HTTP request
2. Tomcat accepts the request, translates and delivers it to the Spring app. The **dispatcher servlet** takes the HTTP request and manages the flow
3. The dispatcher servlet first needs to find out what method of the **Controller** to call depending on the path and the HTTP method of the request. To find the method it delegates the **Handler mapping**
4. Once it knows what method of Controller to call, the dispatcher servlet calls it. After execution, the Controller method provides the view name

53

The Spring MVC architecture



5. The dispatcher servlet finds the view. To find the view to render, the dispatcher servlet delegates a component called **View resolver**
6. Through Tomcat, the HTTP response is returned to the client
7. The browser interprets the HTTP response and displays the data

54

Let's make the acquaintance with a new guy: **Spring Boot**

Everything sounds great, but I'm afraid it's complex, what do you need to do concretely?

Through Spring Boot, Spring arranges the set up of web apps by simplifying it. You only need to write Controller actions and map them to requests using annotations. Most logic is hidden in the framework, so that you can write the apps faster and cleaner



55

Spring Boot: what is it?

- Spring Boot is a project of the Spring ecosystem
- It helps you to write web apps by managing configurations, so that you can focus on the business logic only
- Main features:
 - Simplified project creation through the Spring Initializr
 - Dependency starters
 - Autoconfiguration based on dependencies

56

Simplified project creation

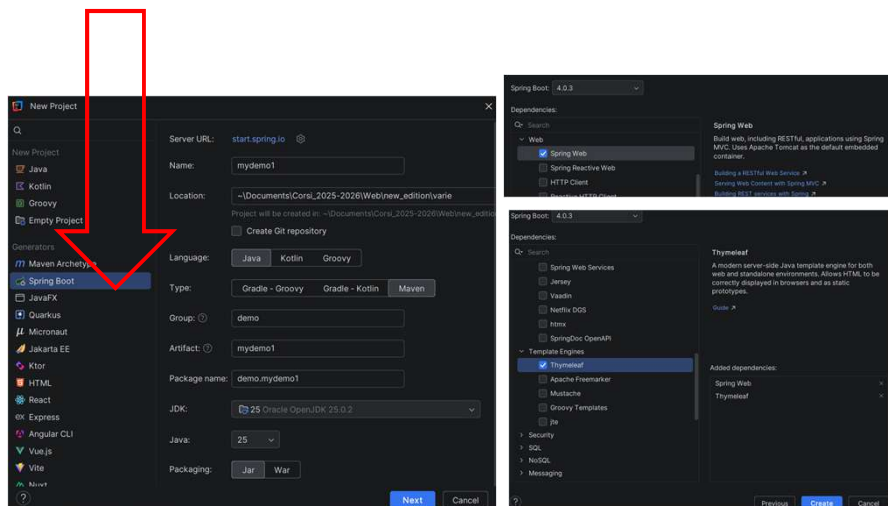
- Spring Initializr allow you to have an empty but already configured skeleton app!
- Some IDEs directly integrate a project initialization service

IntelliJ IDEA Ultimate does it!

Good News!

57

Let's try!



58

What happened?

- The main things Spring Initializr configured into the Maven Project:
The Spring app main class

```
@SpringBootApplication
public class TestApplication {
    public static void main(String[] args) {
        SpringApplication.run(TestApplication.class, args);
    }
}
```

In the pom.xml file:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.2</version>
  <relativePath/>
</parent>
```

The Spring Boot POM Parent

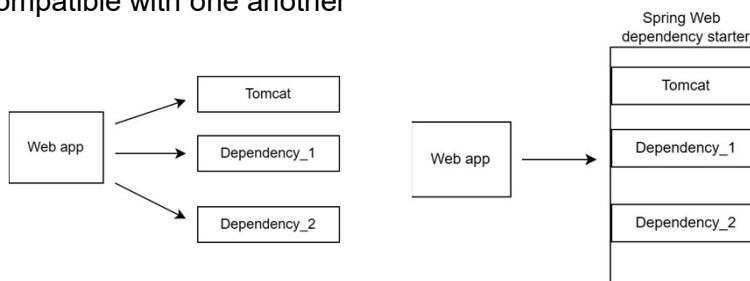
```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

The dependencies

59 ...The Spring Boot Maven plugin and the **properties file** too. We will see it later on!

Dependency starters

- A dependency starter is a group of dependencies added in the pom.xml file to configure a web app for a specific purpose
- Instead of individually referring to specific dependencies, the web app now depends on only a starter containing all the needed dependencies for implementing a specific capability. The starter also makes sure these dependencies are compatible with one another



60

WITHOUT A STARTER

WITH A STARTER

Autoconfiguration based on dependencies

- Spring Boot adopts convention-over-configuration
- The convention represents the most-used way to configure the app for a specific purpose. Spring Boot configures the app by convention such that you now only need to change those places where your app needs a more particular configuration.

Spring Boot configured Tomcat and start it by default on port 8080



```
Tomcat started on port(s): 8080 (http) with context path ''
Started Main in 1.684 seconds (JVM running for 2.306)
```

61

Our first web app with Spring MVC

- Let's keep things simple and focus on a static web app, without any business logic to execute
- We just need:
 - Writing an HTML page with some content to show and store it in the "resources/**templates**" folder
 - Writing a Controller with a method that links the HTTP request to the HTML page to provide in response:

```
@Controller
public class MainController {
    @RequestMapping("/home")
    public String home()
    {return "home"; }
}
```

Create a Bean and annotate it using the stereotype annotation @Controller, a specialization of @Component

Create a method to manage the request, and annotate it using the stereotype annotation @RequestMapping

62

The template engine: Thymeleaf

- Thymeleaf is a server-side Java template engine:
 - **Natural templates**
Template look and work like HTML
 - **Integrations galore**
many IDEs, many frameworks!



<https://www.thymeleaf.org/>

63

Using Thymeleaf

- We added through the Initializr, but if you forgot to do it you can add the right dependency manually:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-thymeleaf</artifactId>  
</dependency>
```

64

Let's try!
esempio6

65

It is not a magic!

```
SpringApplication.run(Application_name.class, args)
```

What happens under the hood:

- SpringBoot starts and it performs:
 - Creation of an instance of the ApplicationContext
 - Component Scanning of beans in the package and the sub-packages of the main application
 - Auto-configuration of beans
 - Start of the application (a embedded Tomcat is created and run in the JVM)
 - Creation of the DispatcherServlet and its registration in Tomcat

66

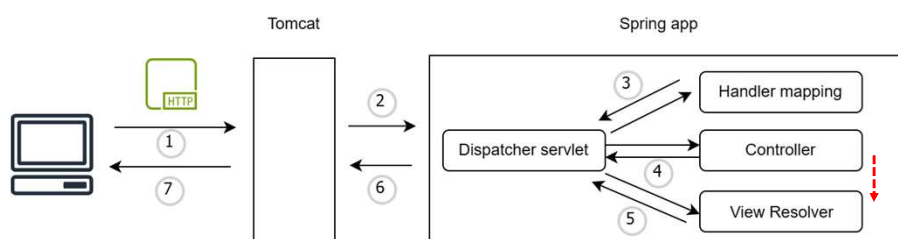
Our problem

what is the current time?



67

How does it work?



4. The Controller returns the view name that the dispatcher servlet needs to find and render into the HTML response. **In addition, the Controller sends data to the view.** The data that is sent can be different for each request. This way, the same view might display different data for each request

68

Let's have a look to the code

```
@Controller
public class MainController {
    @RequestMapping("/home")
    public String home(Model page) {
        DateTimeFormatter day = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        DateTimeFormatter time = DateTimeFormatter.ofPattern("HH:mm:ss");
        LocalDateTime now = LocalDateTime.now();

        page.addAttribute("date", day.format(now));
        page.addAttribute("time", time.format(now));
        return "home.html";
    }
}
```

The action method defines a parameter of type Model that stores the data the controller sends to the view

We add the data we want the controller to send to the view

- The Model object is associated with a specific HTTP request
- A new model is created for any request received by the server
- The Model is local to the called Controller's method
- What about sharing a Model among methods? We will see!

69

What about HTML?

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<p>Hello!</p>
<div> Today is <span th:text="${date}"></span>
and time is <span th:text="${time}"></span></div>
</body>
</html>
```

Defines the th Thymeleaf prefix

Uses the th prefix to use the values sent by the Controller

You need to add the HTML file to the resources/templates folder

70

Let's try!
esempio7

71

let's pass parameters in our request
esempio8

72

Parameters passing

```
<body>
<h1>Enter Your Name and Surname</h1>

<form action="/submit" method="GET">
  <label for="firstName">First Name:</label>
  <input type="text" id="firstName" name="firstName" required><br><br>

  <label for="lastName">Surname:</label>
  <input type="text" id="lastName" name="lastName" required><br><br>

  <input type="submit" value="Submit">
</form>
</body>
```



73

```
@Controller
public class FormHandler {

    @GetMapping("/submit")
    public String receiveFormDatafromGet
        (@RequestParam("firstName") String firstName,
         @RequestParam("lastName") String lastName,
         Model model) {

        model.addAttribute("firstName", firstName);
        model.addAttribute("lastName", lastName);

        return "result"; }

}
```



```
<p>Hello <span th:text="${firstName}">
  </span> <span th:text="${lastName}"></span></p>
```

74

`<form action="/submit" method="POST">`

```

@Controller
public class FormHandler {

    @PostMapping("/submit")
    public String receiveFormDatafromPost
        (@RequestParam("firstName") String firstName,
         @RequestParam("lastName") String lastName,
         Model model) {

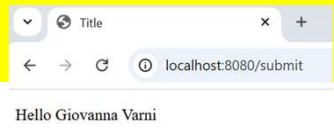
        model.addAttribute("firstName", firstName);
        model.addAttribute("lastName", lastName);

        return "result";}

    }

```





```

<p>Hello <span th:text="\${firstName}">
    </span> <span th:text="\${lastName}"></span></p>

```

75

Web app deployment

- Deploying a web app means making it available to be used on the Internet by moving all what is needs on a server / cloud
- During development, local deployments are made on the Spring Boot's Tomcat for testing and debugging the webapp
- At the end of development, the final version of the web app is moved to the externally accessible web server / cloud to be run

76

Web app deployment

- Two ways:
 - Traditional deployment (through .war):
the web app is copied to a Java application server (Tomcat, Jetty, WildFly). The server runs many apps and provides the web app runtime, lifecycle, and management
 - Modern deployment: (through .jar)
the web app runs as a standalone process or container, and orchestrate with cloud PaaS (Platform-as-a-Service) providers. The web server is embedded in the .jar archive